

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

AGENTS.md

Found in folder: /

This file documents the project's development workflow and expected conventions. It outlines the purpose, working style, git process, validation steps, and communication guidelines for contributors. It serves as a reference for maintaining consistency during development with AI.

AGENTS.md

Repository purpose

This repository is a small demo web app used **for** showing GitHub MCP workflows with Claude Code.

Working style

- Keep changes tightly scoped to the current GitHub issue
- **Do** not refactor unrelated code
- Prefer editing existing files over creating **new** files
- Explain the plan before making code changes
- After implementation, summarize the files changed and the behavior added

Git workflow

- Use feature branch names **in** the form `feature/-`
- Write concise conventional commit messages
- Draft pull request text before creating the PR

Validation

- Run the simplest relevant validation **for** the project
- **For** UI changes, verify behavior **in** the browser
- Mention assumptions or limitations **in** the final summary

Communication

- Be concise and practical
- **Do** not make unrelated improvements
- Ask before making larger structural changes

README.md

Found in folder: /

This code displays the text “Hello, World!” when rendered. It serves as a simple greeting or introductory message. The content is plain text without any formatting.

Hello, World!

SKILL.md

Found in folder: **/.claude/skills/playwright-site-investigator**

This code defines a structured workflow for analyzing public websites using Playwright MCP, focusing on navigation, content discovery, and search behavior. It provides specific guidelines for what to examine and how to report findings without accessing private areas or excessive scraping. The output format requires concise summaries of relevant content locations and discoverability observations.

name: playwright-site-investigator

description: Analyze a public website using Playwright MCP to identify navigation structure, topic discovery paths, visible content areas, and search behavior. Use **this for** public website exploration and structured page analysis.

Playwright Site Investigator

Purpose

Use Playwright MCP to investigate a public website **in** a structured and repeatable way.

This skill is designed **for**:

- understanding homepage structure
- identifying navigation paths
- exploring topic-specific categories

- using on-page search where appropriate
- summarizing findings clearly

Workflow

When **this** skill is used:

1. Open the target public webpage with Playwright MCP
2. Inspect the homepage or landing page before interacting
3. Identify major navigation areas, menus, search fields, and topic discovery elements
4. Look **for** content relevant to the requested topic
5. Use search only **if** it helps clarify discoverability
6. Summarize findings **in** a concise and structured way

Rules

- Focus only on public, visible content
- **Do** not attempt login or private/account areas
- **Do** not dump large raw page content
- **Do** not scrape excessively
- Prefer concise summaries over verbose transcripts
- Explain what was found and where it was found **in** the UI

Output format

Return:

1. a short summary **of** what the site offers related to the topic
2. where the relevant content was found
3. how a **new** user could discover it
4. any observations about clarity or discoverability

File extension: md

file.txt

Found in folder: /

This code outputs the text “Hello, World!” when run. It serves as a basic example program.

Hello, World!

index.html

Found in folder: **/github-mcp-demo-repo**

This HTML file defines the structure of a simple task management web application. It creates a user interface with a form to add new tasks and a section to display and search existing tasks. The page loads external CSS and JavaScript files to handle styling and functionality.

```
<title>TaskBoard Lite</title>

<!-- External stylesheet for application styling -->

<div class="app-shell">

  <header class="app-header">

    <div>

      <p class="eyebrow">Demo Project</p>

      <h1>TaskBoard Lite</h1>

      <p class="subtitle">A tiny task manager for GitHub MCP demos</p>

    </div>

  </header>

  <main class="main-grid">

    <!-- Section containing the form for adding new tasks -->

    <section class="card">

      <h2>Add a Task</h2>

      <label for="task-title">Task title</label>

      <label for="task-category">Category</label>
```

General

Study

Work

Personal

<button type="submit">Add Task</button>

</section>

<!-- Section **for** displaying and searching the list **of** tasks -->

<section **class**="card">

<div **class**="tasks-header">

<div>

<h2>Tasks</h2>

<p **class**="small-text">Manage your current task list</p>

</div>

<div **class**="search-box">

<label **class**="visually-hidden" **for**="search-input">Search tasks</label>

</div>

</div>

<ul id="task-list" **class**="task-list"> <!-- Container where tasks will be dynamically inserted -->

</section>

</main>

</div>

<!-- Main application logic -->

styles.css

Found in folder: **/github-mcp-demo-repo/src**

This CSS styles a task management application with a dark theme, defining the layout, colors, and responsive behavior for the user interface. It sets up a two-column grid for desktop that becomes a single column on mobile, and styles form elements, cards, and a list of tasks. The visual design includes specific spacing, borders, shadows, and states for interactive components like buttons and completed tasks.

```
/* Global box-sizing rule for consistent sizing */
```

```
*{
```

```
  box-sizing: border-box;
```

```
}
```

```
/* Base styles for the page */
```

```
body{
```

```
  margin: 0;
```

```
  font-family: Arial, Helvetica, sans-serif;
```

```
  background: #0f172a;
```

```
  color: #e2e8f0;
```

```
}
```

```
/* Centered container for the main application */
```

```
.app-shell{
```

```
  max-width: 1000px;
```

```
  margin: 0 auto;
```

```
  padding: 32px 20px 60px;
```

```
}
```

```
.app-header{
```

```
  margin-bottom: 24px;
```

```
}
```

```
/* Styling for a small uppercase label */
.eyebrow {
  margin: 0 0 8px;
  text-transform: uppercase;
  letter-spacing: 0.12em;
  font-size: 12px;
  color: #94a3b8;
}

h1 {
  margin: 0;
  font-size: 40px;
}

.subtitle {
  margin-top: 10px;
  color: #cbd5e1;
}

/* Main layout grid: sidebar and content area */
.main-grid {
  display: grid;
  grid-template-columns: 320px 1fr;
  gap: 20px;
}

/* Card component styling */
.card {
  background: #111827;
  border: 1px solid #1f2937;
```

```
border-radius: 16px;

padding: 20px;

box-shadow: 0 12px 30px rgba(0, 0, 0, 0.22);
}

.card h2 {

margin-top: 0;

}

/* Form layout for adding tasks */

.task-form {

display: flex;

flex-direction: column;

gap: 10px;

}

/* Shared input styling for forms and search */

.task-form input,

.task-form select,

.search-box input {

width: 100%;

padding: 12px;

border-radius: 10px;

border: 1px solid #334155;

background: #0b1220;

color: #e2e8f0;

}

/* Primary action button */

.task-form button,
```



```
.delete-btn {  
  padding: 10px 14px;  
  border: none;  
  border-radius: 10px;  
  background: #3b82f6;  
  color: white;  
  cursor: pointer;  
  font-weight: 600;  
}
```

```
/* Destructive action button */
```

```
.delete-btn {  
  background: #ef4444;  
}
```

```
/* Header area for the tasks list with controls */
```

```
.tasks-header {  
  display: flex;  
  justify-content: space-between;  
  gap: 16px;  
  align-items: end;  
  margin-bottom: 16px;  
}
```

```
.small-text {  
  margin: 6px 0 0;  
  color: #94a3b8;  
  font-size: 14px;  
}
```

```
/* List container for tasks */
```

```
.task-list {  
  list-style: none;  
  padding: 0;  
  margin: 0;  
  display: flex;  
  flex-direction: column;  
  gap: 12px;  
}
```

```
/* Individual task item layout */
```

```
.task-item {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  gap: 12px;  
  padding: 14px;  
  border: 1px solid #273449;  
  border-radius: 12px;  
  background: #0b1220;  
}
```

```
.task-left {  
  display: flex;  
  align-items: flex-start;  
  gap: 12px;  
}
```

```
.task-title {
```

```
margin: 0;

font-weight: 700;
}

/* Style for a completed task title */
.task-title.completed {

text-decoration: line-through;

color: #94a3b8;
}

.task-meta {

margin: 6px 0 0;

color: #94a3b8;

font-size: 14px;
}

/* Utility class to hide content visually but keep it accessible */
.visually-hidden {

position: absolute;

left: -9999px;
}

/* Responsive adjustments for smaller screens */
@media (max-width: 800px) {

.main-grid {

grid-template-columns: 1fr;
}

.tasks-header {

flex-direction: column;

align-items: stretch;
```

```
}  
}
```

app.js

Found in folder: **/github-mcp-demo-repo/src**

This code manages a task list application. It loads tasks from local storage or initial data, renders them as interactive list items, and handles adding, toggling completion, deleting, and searching tasks. User interactions update the task list and persist changes to local storage.

```
import { initialTasks } from "./data.js";  
  
// DOM element references  
  
const taskForm = document.getElementById("task-form");  
const taskTitleInput = document.getElementById("task-title");  
const taskCategorySelect = document.getElementById("task-category");  
const taskList = document.getElementById("task-list");  
const searchInput = document.getElementById("search-input");  
const STORAGE_KEY = "tasks";  
  
// Load tasks from localStorage or use initial data  
  
function loadTasks() {  
  const saved = localStorage.getItem(STORAGE_KEY);  
  return saved ? JSON.parse(saved) : [...initialTasks];  
}  
  
// Save current tasks array to localStorage  
  
function saveTasks() {  
  localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));  
}  
  
let tasks = loadTasks();  
let searchTerm = "";
```

```
// Render the task list based on current tasks and search filter

function renderTasks() {

  // Filter tasks by search term matching title

  const filteredTasks = tasks.filter((task) =>

    task.title.toLowerCase().includes(searchTerm.toLowerCase())

  );

  taskList.innerHTML = "";

  // Create a list item for each filtered task

  filteredTasks.forEach((task) => {

    const li = document.createElement("li");

    li.className = "task-item";

    const leftSide = document.createElement("div");

    leftSide.className = "task-left";

    const checkbox = document.createElement("input");

    checkbox.type = "checkbox";

    checkbox.checked = task.completed;

    checkbox.addEventListener("change", () => toggleTask(task.id));

    const content = document.createElement("div");

    const title = document.createElement("p");

    title.className = task.completed ? "task-title completed" : "task-title";

    title.textContent = task.title;

    const meta = document.createElement("p");

    meta.className = "task-meta";

    meta.textContent = `Category: ${task.category}`;

    content.appendChild(title);

    content.appendChild(meta);
```

```

    leftSide.appendChild(checkbox);
    leftSide.appendChild(content);
    const deleteButton = document.createElement("button");
    deleteButton.className = "delete-btn";
    deleteButton.textContent = "Delete";
    deleteButton.addEventListener("click", () => deleteTask(task.id));
    li.appendChild(leftSide);
    li.appendChild(deleteButton);
    taskList.appendChild(li);
  });
}

// Add a new task to the beginning of the tasks array
function addTask(title, category) {
  const newTask = {
    id: Date.now(), // Use timestamp for unique ID
    title,
    category,
    completed: false
  };
  tasks.unshift(newTask);
  saveTasks();
  renderTasks();
}

// Toggle the completed status of a task by ID
function toggleTask(taskId) {
  tasks = tasks.map((task) =>

```

```

    task.id === taskId

    ? { ...task, completed: !task.completed }

    : task
  );
  saveTasks();
  renderTasks();
}

// Remove a task by ID
function deleteTask(taskId) {
  tasks = tasks.filter((task) => task.id !== taskId);
  saveTasks();
  renderTasks();
}

// Form submission handler for adding new tasks
taskForm.addEventListener("submit", (event) => {
  event.preventDefault();

  const title = taskTitleInput.value.trim();
  const category = taskCategorySelect.value;

  if (!title) {
    alert("Please enter a task title.");
    return;
  }

  addTask(title, category);

  taskForm.reset();
});

// Search input handler to filter tasks

```

```
searchInput.addEventListener("input", (event) => {  
  searchTerm = event.target.value;  
  renderTasks();  
});  
  
// Initial render  
renderTasks();
```

data.js

Found in folder: **/github-mcp-demo-repo/src**

This code exports an array of three task objects with properties for id, title, category, and completion status. It provides sample data that can be imported and used throughout the application. The tasks represent different categories and completion states for demonstration purposes.

```
// Export an array of initial task objects for use throughout the application
```

```
export const initialTasks = [  
  {  
    id: 1,  
    title: "Prepare GitHub MCP demo",  
    category: "Work",  
    completed: false  
  },  
  {  
    id: 2,  
    title: "Review Claude Code prompts",  
    category: "Study",  
    completed: true  
  },  
  {
```



```
id: 3,  
title: "Plan next video outline",  
category: "Personal",  
completed: false  
}  
];
```

README.md

Found in folder: **/github-mcp-demo-repo**

This Markdown file provides an overview of the TaskBoard Lite project, describing its purpose as a demo for GitHub MCP workflows and listing its features, tech stack, and local setup instructions.

TaskBoard Lite

TaskBoard Lite is a very small vanilla JavaScript web app **for** managing simple tasks.

This repository is designed as a demo project **for** showcasing real GitHub MCP workflows with Claude Code.

Features

- Add tasks
- Search tasks
- Mark tasks as complete
- Delete tasks
- Small and beginner friendly codebase

Tech Stack

- HTML
- CSS
- Vanilla JavaScript

Run Locally

You can use any static file server. One simple option is:

```
```bash
npm install
npm run start
```
```